Preprocessing for SignBase Sequences

Description

This file enables preprocessing of Upper-Paleolithic sign sequences as published at https://www.signbase.org. The newest csv file with updated objects and sign codings, as well as a csv file with the sign type typology can be downloaded from there. The files used for the current analyses published in this article are given in the github repo under data/signBase/.

Randomization

The code below also enables randomization of sign strings which occur on different parts of a paleolithic object (indicated by underscore '_'). In the case of paleolithic sequences, we do not know the intended order of these strings of signs (if any). To understand the impact of string order we can randomize strings and compare the results to the original (arbitrary) order. Note that this is not applicable to modern day written languages, as the order of strings of characters (e.g. words) is normally meaningful. Randomization of words would artificially introduce randomness where order is intended. Arguably, the same holds for proto-cuneiform writing. There are particular orders with which proto-cuneiform signs were imprinted onto a tablet, i.e. a kind of syntax, cf. the following statement in Englund (1998), p. 79:

"Number sign sequences within discrete notations are, as might be expected, very rigid and follow a definite numerical 'syntax'. Within text entries, moreover, the position of numerical notations relative to ideographic notations is fairly rigid."

Again, randomization would destroy this syntax and artificially introduce randomness where order was intended by the scribes.

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"
```

```
## [1] "x86_64-pc-linux-gnu"
```

Load Data

List of Sign Types

```r
# load data from github repo
# sign typology
sign.typology <- read.csv("~/Github/PaleoSigns/data/signBase/Sign-2024-07-17.csv")
#print(sign.typology[,1:4])
```

Coding of Sign Sequences

```r
# sign codings and meta info per object
sign.data <- read.csv("~/Github/PaleoSigns/data/signBase/Sb2Artifact-2024-11-21.csv")
# select objects of the Swabian Aurignacien
sign.data <- sign.data[sign.data$archaeological_unit == "Aurignacien" &
                      sign.data$site_name == "Vogelherd" |
                      sign.data$site_name == "Hohle Fels" |
                      sign.data$site_name == "Geissenklösterle" |
                      sign.data$site_name == "Hohlenstein-Stadel" |
                      sign.data$site_name == "Bockstein-Törle", ]
# display the head of some columns of the data
head(subset(sign.data, select = c("object_id", "archaeological_unit", "site_name", "sign_coding")))
```

```
##     object_id archaeological_unit       site_name             sign_coding
## 91    bst0003         Aurignacien Bockstein-Törle
## 92    bst0002         Aurignacien Bockstein-Törle
## 162   vhc0168         Aurignacien       Vogelherd         >>>>>> ! >>>>>
## 163   vhc0167         Aurignacien       Vogelherd vvvvvvvvvvvvvvvvvv _ v!
## 164   vhc0166         Aurignacien       Vogelherd           vvvvvvv _ [&]
## 165   vhc0165         Aurignacien       Vogelherd   !vvvv v v v vv v >>>>
```

Simple Stats (before Preprocessing)

```r
# number of sequences/objects
nrow(sign.data)
```

```
## [1] 260
```

```r
# cave sites
table(sign.data$site_name)
```

```
##
##    Bockstein-Törle   Geissenklösterle         Hohle Fels Hohlenstein-Stadel
##                  3                 30                 50                  2
##          Vogelherd
##                175
```

```r
# material
table(sign.data$material)
```

```
##
## animal tooth       antler         bone       fossil        ivory        ochre
##            1            7           76            1          172            1
##    sandstone        slate
##            1            1
```

```r
# object_type
table(sign.data$object_type)
```

```
##
##          antler object                     awl             bâton percé
##                      3                       6                       4
##               figurine figurine anthropomorph      figurine zoomorph
```

```
##                   8                 6                25
##               flute personal ornament         plaquette
##                  10                18                 1
##               point possible figurine    possible flute
##                   1                24                15
##           rod/bâton   spatula/lissoir              tube
##                  20                12                 1
##          tube/flute      undetermined
##                  24                82
```

Preprocessing

Preprocessing and storage of cleaned file. Load data from github repo and remove missing rows.

```r
# remove certain columns
sign.data <- subset(sign.data, select = -c(short_description, references))
# remove rows for which there is no sign coding
sign.data <- sign.data[!sign.data$sign_coding == "", ]
# remove rows with "very uncertain" sign codings
sign.data <- sign.data[!sign.data$coding_certainty == "Very uncertain", ]
```

Remove UTF-8 characters which do not represent actual signs on the objects, i.e annotations. See also list of sign types (sign.typology) above.

```r
# remove some special characters
coding.clean <- gsub('[!?{}]', '', sign.data$sign_coding)
# remove sign strings which are not discrete, i.e. overlapping sign types
# -> indicated by parentheses '()'
coding.clean <- gsub("\\([^()]*\\)", "", coding.clean)
# remove sign strings which are not in linear order, i.e. those grouped in visual fields
# -> indicated by square brackets '[]'
coding.clean <- gsub("\\[(.*?)\\]", "", coding.clean)
# replace multiple underscores by single underscore
coding.clean <- gsub("( _ )+", " _ ", coding.clean)
# trim the underscores and white spaces from beginning and end of sequences
coding.clean <- trimws(coding.clean, which = "both", whitespace = "[ \\_]")
# remove white spaces around underscores
coding.clean <- gsub(" _ ", "_", coding.clean)

# add column to original data frame again
sign.data$coding_clean <- coding.clean
# again remove rows for which there is no sign coding (after cleaning)
sign.data <- sign.data[!sign.data$coding_clean == "", ]
```

```r
# use only lines with more than one character
sign.data <- sign.data[nchar(sign.data$coding_clean) > 1, ]
```

Simple Stats (after Preprocessing)

```r
# number of sequences/objects
nrow(sign.data)
```

```
## [1] 211
```

```r
# cave sites
table(sign.data$site_name)
```

```
##
##    Bockstein-Törle   Geissenklösterle          Hohle Fels Hohlenstein-Stadel
##                  1                 28                  33                  2
##          Vogelherd
##                147
```

```r
# material
table(sign.data$material)
```

```
##
## animal tooth       antler         bone       fossil        ivory    sandstone
##            1            6           67            1          134            1
##        slate
##            1
```

```r
# object_type
table(sign.data$object_type)
```

```
##
##          antler object                    awl            bâton percé
##                      3                      4                      3
##               figurine figurine anthropomorph      figurine zoomorph
##                      6                      6                     21
##                  flute      personal ornament              plaquette
##                      8                     16                      1
##      possible figurine         possible flute             rod/bâton
##                     20                     15                     15
##        spatula/lissoir                   tube            tube/flute
##                     10                      1                     20
##           undetermined
##                     62
```

Randomization

Randomize sign sequences per object.

```r
# set seed to get reproducible result
set.seed(09012020)
# initialize empty vector
coding.random <- c()
# run loop to randomize, and add new column to data frame
for (i in 1:nrow(sign.data)){
  # use single coding per row
  coding.local <- sign.data[i, ]$coding_clean
  # split on underscores surrounded by white spaces
  coding.split <- unlist(strsplit(as.character(coding.local), split = "_"))
  # randomize order
  coding.random.local <- sample(coding.split)
  # concatenate with underscore and white space again
  coding.random.local <- paste(coding.random.local, collapse = "_")
```

```
  # add to vector
  coding.random <- c(coding.random, coding.random.local)
}

# add column with randomized order to data frame
sign.data$coding_random <- coding.random
```

Safe to file.

```
write.csv(sign.data, "~/Github/PaleoSigns/data/signBase/signBase_randomized.csv", row.names = F)
```

Literature References

Englund (1998). Texts from the Late Uruk period. In Bauer et al. (Eds.), Späturuk-Zeit und Frühdynastische Zeit.